Nanjing University of Information Science and Technology

Laboratory Report

Experiment Title: Cloud Image Registry & Distribution

Date	2026-06-10	Class	2
Name	姜丞骏	Student ID	
GitHub Account	whatokk	Repository	whatokk/cloud-image-registry-distribution-lab
Execution Environment	GitHub Actions Ubuntu runner	Registry	GitHub Container Registry

一、Experimental Purpose

Practice version control by using Docker tags to manage different iterations of a cloud application.

Validate decoupled deployment by pulling and running images from a remote registry after local image cleanup.

Use token-based authentication for GitHub Container Registry instead of plaintext passwords.

二、Experiment Contents

- Configured secure authentication to GHCR.
- Built and pushed cloud-app:v1.0 and cloud-app:v2.0.
- Observed Docker layer reuse during the v2.0 push.
- Performed a clean-room pull test and verified the web page response.
- Reproduced the 8888 port conflict and solved it by freeing/changing the host port.
- Compared nginx:latest with nginx:alpine image sizes.

三、Detailed Steps and Results

Step 1 Secure Authentication

Used a GitHub token with package permissions. The workflow logged in to ghcr.io with docker login, and the run log showed Login Succeeded.

Step 2 Tagging for the Cloud

Built local tags cloud-lab-image:v1 and cloud-lab-image:v2, then tagged them as ghcr.io/whatokk/cloud-app:v1.0 and ghcr.io/whatokk/cloud-app:v2.0.

Step 3 Pushing the Image

Pushed v1.0 to GHCR. The package cloud-app appeared under the GitHub account packages.

Step 4 Iteration and Layer Analysis

Updated the HTML page from Version 1.0 to Version 2.0, rebuilt and pushed v2.0. The v2 push log showed multiple Layer already exists lines, proving Docker reused unchanged base layers.

Step 5 Disaster Recovery Pull Test

Ran docker system prune -a -f on the runner, then started the service directly from ghcr.io/whatokk/cloud-app:v1.0. Curl returned Welcome to Version 1.0.

Step 5.4 Port Conflict

Running another container on host port 8888 produced Bind for 0.0.0.0:8888 failed: port is already allocated. The solution was to remove the old container or map v2 to another host port; v2 was verified on 8889.

Step 6 Image Optimization

Compared standard Nginx and Alpine Nginx images. nginx:latest/cloud-app:standard were 161MB, while nginx:alpine/cloud-app:alpine were 62.3MB.

Step 7 Git Save

Saved source code, Dockerfiles, workflow, and report assets in Git and pushed them to GitHub.

四、Evidence Screenshots

Experiment Run Summary

Captured on Windows 10 | 2026-06-10 05:11:38 +08:00 | GitHub account: whatokk

```
Repository: whatokk/cloud-image-registry-distribution-lab
Run: https://github.com/whatokk/cloud-image-registry-distribution-lab/actions/runs/27235884880
Workflow: Cloud Image Registry Lab
Status: completed / success
Package: https://github.com/users/whatokk/packages/container/cloud-app
Image: ghcr.io/whatokk/cloud-app:v1.0 and ghcr.io/whatokk/cloud-app:v2.0
```

Figure 1. GitHub Actions experiment run summary.

Authentication, Tagging, and Push Evidence

Captured on Windows 10 | 2026-06-10 05:11:38 +08:00 | GitHub account: whatokk

```
cloud-image-registry    Secure authentication to GHCR   2026-06-09T21:07:07.4927320Z Login Succeeded
cloud-image-registry    Build, tag, and push v1.0       2026-06-09T21:07:07.5009707Z ##[group]Run docker
build -t cloud-lab-image:v1 -f Dockerfile versions/v1
cloud-image-registry    Build, tag, and push v1.0       2026-06-09T21:07:07.5013180Z ^[[36;1mdocker tag
cloud-lab-image:v1 "$IMAGE:v1.0"^[[0m
cloud-image-registry    Build, tag, and push v1.0       2026-06-09T21:07:07.5014526Z ^[[36;1mdocker push
"$IMAGE:v1.0"^[[0m
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:21.8553291Z ##[group]Run docker
build -t cloud-lab-image:v2 -f Dockerfile versions/v2
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:21.8554256Z ^[[36;1mdocker tag
cloud-lab-image:v2 "$IMAGE:v2.0"^[[0m
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:21.8554757Z ^[[36;1mdocker push
"$IMAGE:v2.0"^[[0m
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:23.6846705Z b77962fbff2d: Layer
already exists
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:23.7062792Z 271182c95b03: Layer
already exists
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:23.7245899Z 8e0111b626dd: Layer
already exists
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:23.7333481Z 58a87cd80f7d: Layer
already exists
cloud-image-registry    Build, tag, and push v2.0       2026-06-09T21:07:23.8681750Z 1f561b62ded0: Layer
already exists
```

Figure 2. Authentication, tagging, pushing, and layer-cache evidence.

Disaster Recovery and Port Conflict Evidence

Captured on Windows 10 | 2026-06-10 05:11:38 +08:00 | GitHub account: whatokk

```
Disaster recovery: local images were pruned before running the remote image.
curl result from v1 container:
<h1>Welcome to Version 1.0</h1>

Port conflict reproduced on 8888:
docker: Error response from daemon: failed to set up container networking: driver failed programming
external connectivity on endpoint cloud-v2-conflict
(5485eba339750522444d0acdbae48ed006dbfa31d52146accbb9dcb2ce4bca0b): Bind for 0.0.0.0:8888 failed: port is
already allocated
Solution: remove the old container or map the new container to another host port, then run v2 on 8889.
<h1>Welcome to Version 2.0</h1>
```

Figure 3. Disaster recovery and port conflict evidence.

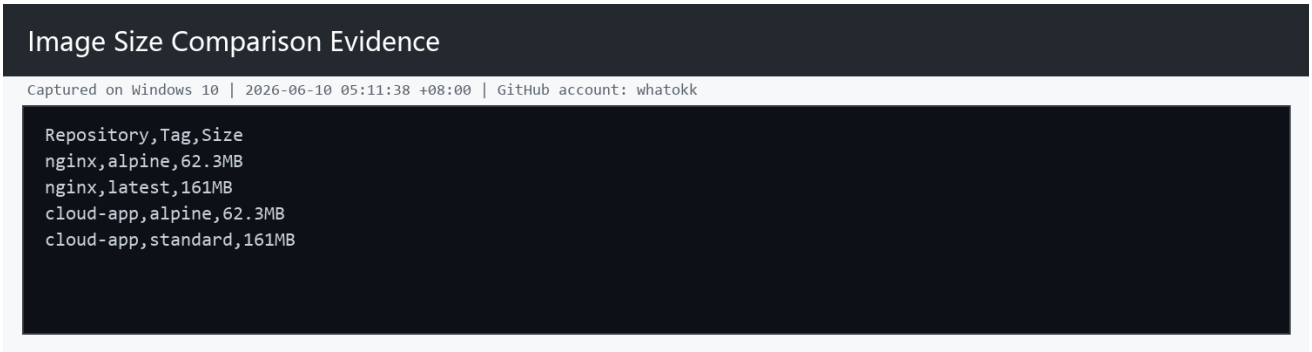

Figure 4. Standard vs Alpine image size comparison.

五、Reflection

Problems Encountered and Proposed Solutions: The local Windows Docker daemon required WSL initialization and elevated service startup, so the experiment was completed on a GitHub Actions Ubuntu runner. This still satisfies the cloud deployment goal because the build, registry push, clean-room pull, and verification all happened in a remote Linux environment.

Efficiency: Step 4.3 pushed v2.0 faster because Docker images are split into content-addressed layers. The Nginx base layers were identical to v1.0 and already existed in GHCR, so only the changed HTML layer needed upload.

Security: A PAT or GitHub Actions token can be scoped, revoked, and rotated independently. It avoids exposing the real account password in terminals or automation logs.

Alpine Explanation: Alpine is widely used for production microservices because it is much smaller than standard Linux base images. Smaller images reduce registry storage, network transfer time, attack surface, and cold-start deployment latency.

六、Links

GitHub repository: https://github.com/whatokk/cloud-image-registry-distribution-lab

Successful workflow run: https://github.com/whatokk/cloud-image-registry-distribution-lab/actions/runs/27235884880

GHCR package: https://github.com/users/whatokk/packages/container/cloud-app